UT CHETUMAL

Desarrollo y Gestión de Software

Extracción de conocimiento en bases de datos

Dr. Martín Antonio Santos Romero

T.S.U. Miguel Ángel Peña Pech

9no cuatrimestre

Práctica guiada 5

Algoritmo de clasificación

Índice

Introducción 3

Tareas a realizar por el alumno 4

Conclusión 11

Introducción

En esta actividad se implementaron diferentes algoritmos de clasificación utilizando el lenguaje Python y la biblioteca Scikit-Learn, aplicados al clásico conjunto de datos Iris. El objetivo principal fue comprender el flujo de trabajo de un proyecto de aprendizaje supervisado, abarcando desde la importación y exploración de datos, separación de variables predictoras y objetivo, división en conjuntos de entrenamiento y prueba, hasta la construcción, evaluación y comparación de modelos de clasificación como K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Regresión Logística, Árboles de Decisión y Naive Bayes. Además, se exploró el uso de GridSearchCV para la optimización de hiperparámetros, permitiendo ajustar de forma eficiente el rendimiento de los modelos seleccionados. Este proceso permitió reforzar las competencias en el uso de herramientas de análisis y clasificación de datos, así como la interpretación de resultados para su posterior aplicación en la resolución de problemas de clasificación con datos reales.

Tareas a realizar por el alumno

1: Utilizando Jupyter Notebook realiza los ejercicios (laboratorios) del Módulo I Módulos, paquetes y PIP; del curso Fundamentos de Python Curso 2:

-Crea un nuevo proyecto en Jupyter.
-Guardarlo con el nombre ejerciciosModuloV_NombreApellidos
-Por cada ejercicio crea una celda nueva de código.

Se pretende clasificar especies de flores Iris, con base a las siguientes variables de entrada y variable objetivo:

Variables de entrada (atributos):

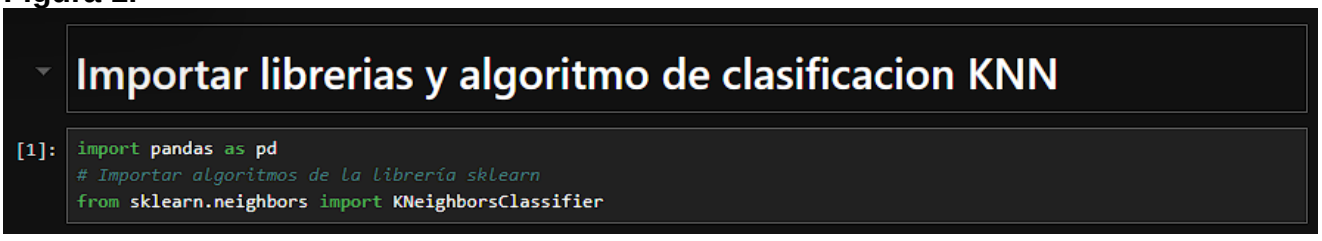
-Longitud pétalo
-Ancho pétalo
-Longitud sépalo
-Ancho sépalo

Variable objetivo (especie):

-Versicolor
-Virgínica
-Setosa

Se importan las herramientas necesarias para el análisis. pandas se utiliza para manipular y analizar datos estructurados en forma de tablas (DataFrames). Luego, se importa el algoritmo KNeighborsClassifier desde la biblioteca scikit-learn, que permite aplicar el algoritmo de clasificación KNN (K-Nearest Neighbors). Este algoritmo clasifica instancias nuevas basándose en las clases de sus vecinos más cercanos.

Figura 1.



Fuente: Angel Peña (2025).

Se carga un archivo Excel que contiene los datos de entrada (aparentemente del famoso dataset de Iris). pd.read_excel lee el archivo y lo convierte en un DataFrame. Luego, df_data.head() permite visualizar las primeras cinco filas del conjunto de datos, lo cual es útil para verificar que los datos se cargaron correctamente y para familiarizarse con su estructura.

Figura 2.

Crear el dataset con la base de datos

```
[2]: df_data = pd.read_excel(r'C:\Users\king1\tareasPython\dataset practica guiada 5\iris.xlsx')
     df_data.head()
```

[2]:

	long_sepalo	anch_sepalo	long_petalo	anch_petalo	clase
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Fuente: Angel Peña (2025).

Se separan las variables independientes (características o atributos) de la variable dependiente (etiqueta o clase). X contiene todas las columnas del DataFrame excepto la columna clase, que es la que se desea predecir. y contiene únicamente esa columna clase. .values convierte los datos en arreglos NumPy, lo cual es el formato requerido por los modelos de scikit-learn.

Figura 3.

Crear arrays con variables indeprendientes y variables dependientes

```
[3]: X = df_data.drop('clase', axis=1).values
     y = df_data['clase'].values
```

Fuente: Angel Peña (2025).

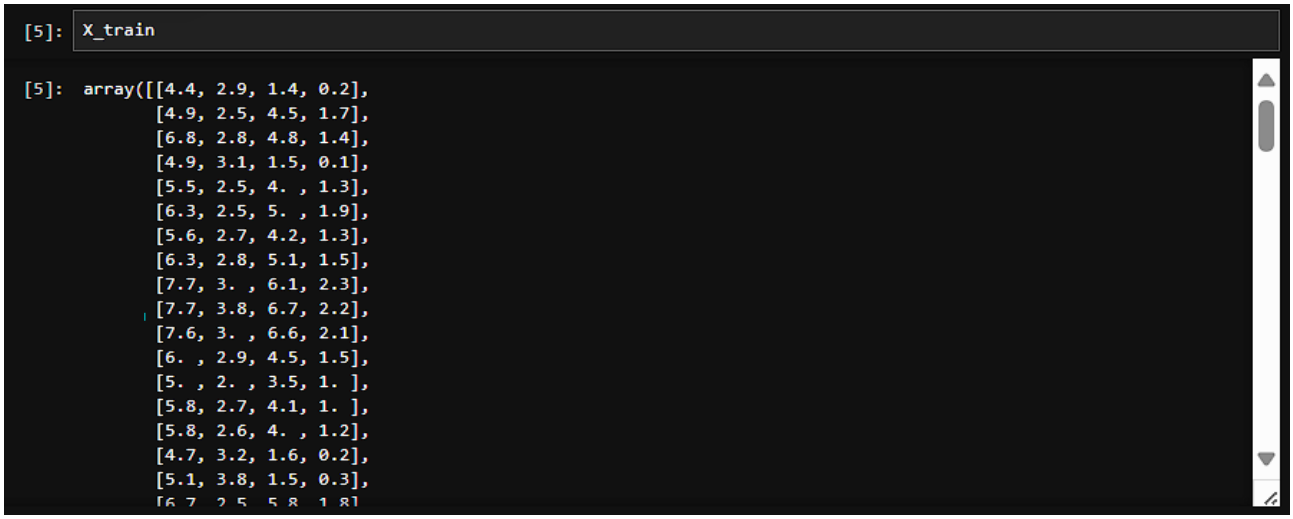
Se divide el conjunto de datos en dos subconjuntos: uno para entrenar el modelo (X_train, y_train) y otro para probarlo (X_test, y_test). Se utiliza train_test_split, especificando que el 20% de los datos se usará para pruebas (test_size=0.2). El parámetro random_state=42 asegura que la división sea reproducible. Además, se utiliza stratify=y para garantizar que ambas partes mantengan la misma proporción de clases, lo cual es importante para que el modelo sea entrenado de manera equilibrada.

Figura 4.

Dividir los datos en entrenamiento (80%) y prueba (20%)

```
[4]: from sklearn.model_selection import train_test_split

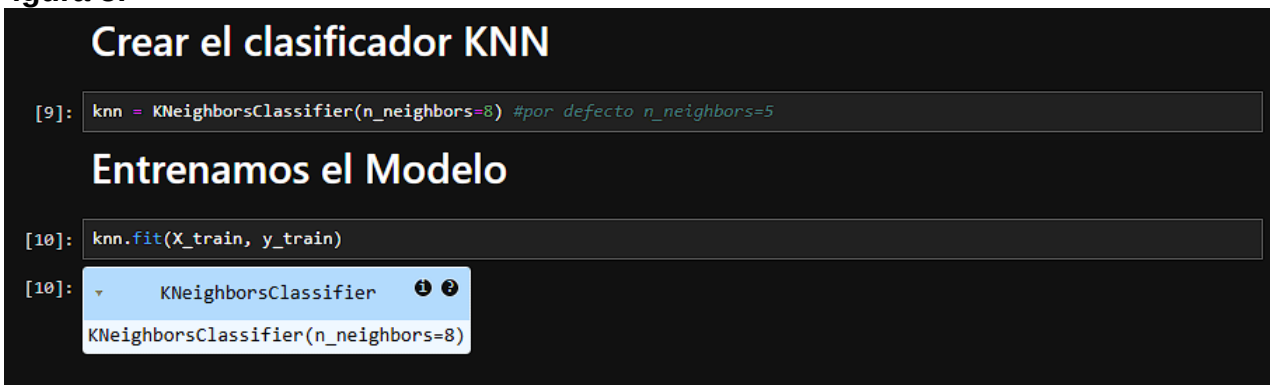
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42, stratify=y) #stratify -->
```



Fuente: Angel Peña (2025).

Se inicializa un clasificador KNN (K-Nearest Neighbors) con n_neighbors=8 y se entrena con los datos de entrenamiento. KNN es un algoritmo de clasificación que asigna la clase según la mayoría de vecinos cercanos en el espacio de características.

Figura 5.



Fuente: Angel Peña (2025).

Se calcula la precisión del modelo usando .score() sobre el conjunto de prueba (20% de los datos), mostrando el rendimiento inicial sin validación cruzada, lo que permite obtener una primera métrica de desempeño del modelo entrenado.

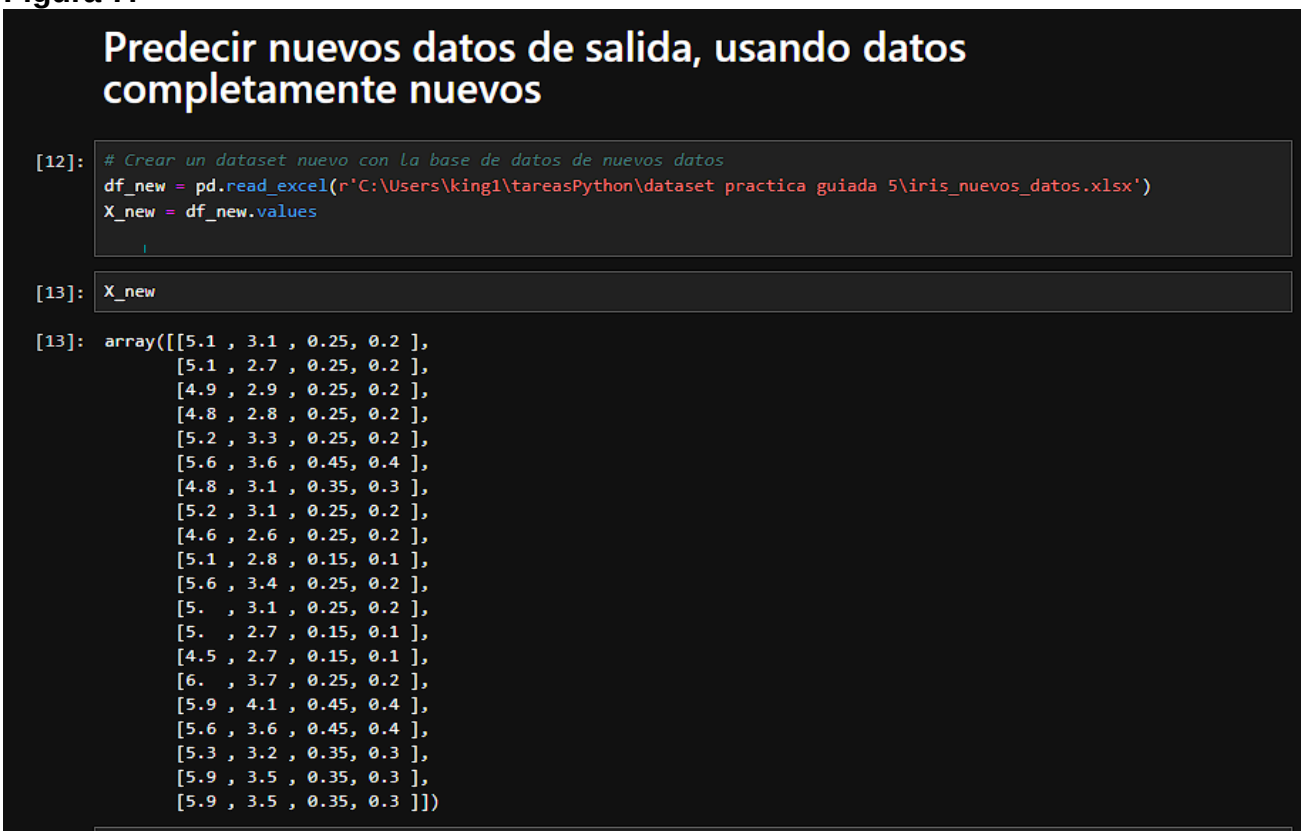
Figura 6.



Fuente: Angel Peña (2025).

Se cargan nuevos datos desde iris_nuevos_datos.xlsx, se procesan, y se utiliza el modelo KNN previamente entrenado para predecir sus clases. Se imprime el arreglo de clases predichas para observar el resultado de la predicción sobre datos reales.

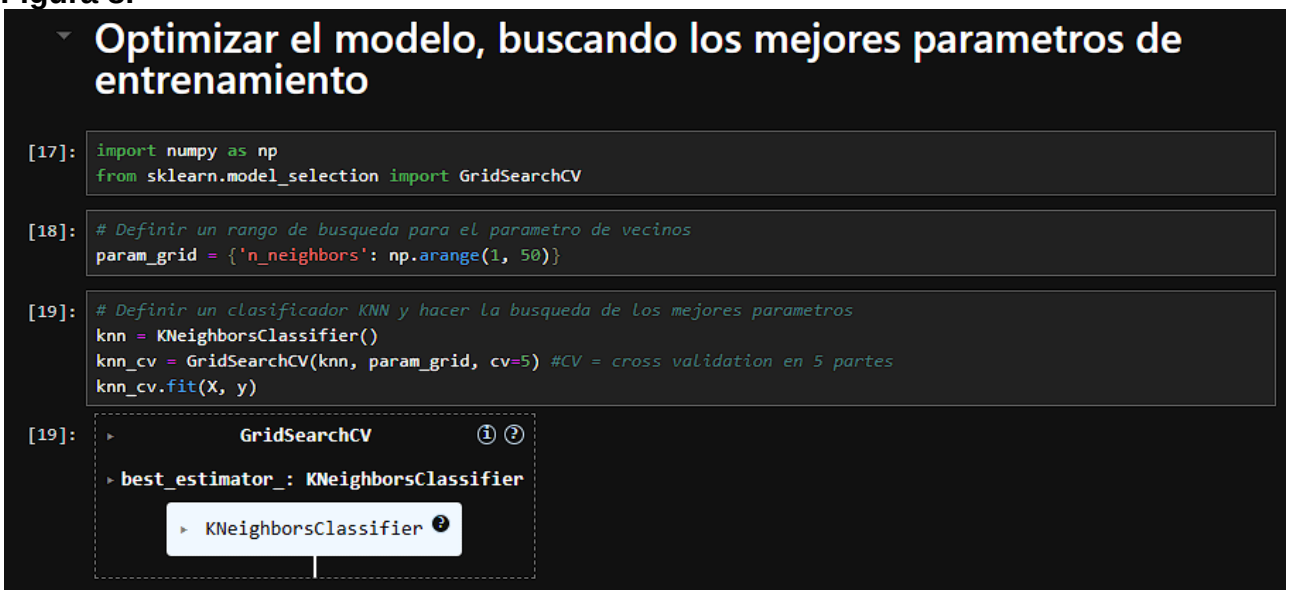
Figura 7.



Fuente: Angel Peña (2025).

Se define un rango de valores para n_neighbors de 1 a 50 y se usa GridSearchCV con validación cruzada de 5 particiones (cv=5) para encontrar el número óptimo de vecinos que maximice la precisión del modelo en KNN. Esto optimiza el modelo de forma automatizada.

Figura 8.



Fuente: Angel Peña (2025).

best_params_: el mejor valor encontrado para n_neighbors.
best_score_: la mejor puntuación media de validación cruzada alcanzada.
Esto ayuda a justificar la elección del mejor hiperparámetro con base en resultados objetivos.

Figura 9.

```
[21]: # Visualizar el mejor parametro
      knn_cv.best_params_

[21]: {'n_neighbors': 6}

[22]: # Visualiza el mejor rendimiento
      knn_cv.best_score_

[22]: 0.9800000000000001
```

Fuente: Angel Peña (2025).

Se importa LogisticRegression, DecisionTreeClassifier, KNeighborsClassifier, GaussianNB, SVC y se agrupan en una lista para comparar su rendimiento. Usando cross_val_score con validación cruzada de 10 particiones, se evalúa cada modelo y se imprime su precisión media y desviación estándar para comparar su desempeño de manera justa.

Figura 10.

Comparacion de algoritmos de clasificacion

```
[23]: # Importacion de librerias
      from pandas.plotting import scatter_matrix
      import matplotlib.pyplot as plt
      from sklearn import model_selection
      from sklearn.metrics import classification_report
      from sklearn.metrics import confusion_matrix
      from sklearn.metrics import accuracy_score
      from sklearn.linear_model import LogisticRegression
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
      from sklearn.naive_bayes import GaussianNB
      from sklearn.svm import SVC
```

Importar datos y crear el dataset

```
[24]: df_data = pd.read_excel(r'C:\Users\king1\tareasPython\dataset practica guiada 5\Iris.xlsx')
      df_data.head()
```

[24]:

	long_sepalo	anch_sepalo	long_petalo	anch_petalo	clase
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
[25]: # Crear los arrays con la variables de entrada y variables de salida
      X = df_data.drop('clase', axis=1).values
      y = df_data['clase'].values
```

```
[26]: # Dividir los datos en conjunto de entrenamiento y conjunto de prueba
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42, stratify=y) #stratify -->

[27]: # Construir todos los modelos con base en los diferentes algoritmos

models = []
models.append(('LR', LogisticRegression()))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC()))

[29]: # Evaluar los modelos creando dos arrays
results = []
names = []

for name, model in models:
    kfold = model_selection.KFold(n_splits=10, random_state=42, shuffle=True)
    cv_results = model_selection.cross_val_score(model, X, y, cv=kfold) #No sería inclusonecesaria haber separado pre
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)

LR: 0.973333 (0.044222)
LDA: 0.980000 (0.030551)
KNN: 0.973333 (0.044222)
CART: 0.940000 (0.055377)
NB: 0.960000 (0.053333)
SVM: 0.966667 (0.044721)
```

Fuente: Angel Peña (2025).

se selecciona de forma específica el modelo SVC (Support Vector Classifier) de Scikit-Learn, el cual es útil para problemas de clasificación donde se busca encontrar el hiperplano óptimo que separe las clases. Este paso prepara el modelo SVM para su posterior ajuste de hiperparámetros.

Figura 11.

```
[30]: # Seleccionar un modelo en especifico
svc = SVC()
```

Fuente: Angel Peña (2025).

se optimiza el modelo SVM mediante GridSearchCV, definiendo un rango de valores para los hiperparámetros C y gamma. Se realiza una validación cruzada de 5 particiones (cv=5) para encontrar automáticamente la combinación que brinde el mejor rendimiento en el conjunto de datos. De esta forma, se ajusta el modelo SVM para lograr una clasificación más precisa, asegurando un desempeño óptimo en comparación con los valores predeterminados.

Figura 12.

```
[31]: # Optimzar el modelo para buscar sus mejores parametros
      import numpy as np
      from sklearn.model_selection import GridSearchCV

      Cs = [0.001, 0.01, 0.1, 1, 10]
      gammas = [0.001, 0.01, 0.1, 1]
      param_grid = {'C': Cs, 'gamma' : gammas}
      svc_cv = GridSearchCV(svc, param_grid, cv=5)
      svc_cv.fit(X, y)

[31]: GridSearchCV
      best_estimator_: SVC
      SVC
```

Fuente: Angel Peña (2025).

Se visualiza la salida de la optimización con GridSearchCV, mostrando los valores óptimos de C y gamma encontrados (best_params_) y la mejor puntuación de precisión promedio obtenida durante la validación cruzada (best_score_). Esto confirma que el modelo ha sido afinado correctamente antes de realizar predicciones sobre nuevos datos.

Figura 13.

```
[34]: #Visualizar los mejores parametros
      svc_cv.best_params_

[34]: {'C': 1, 'gamma': 0.1}

[33]: # Visualizar los mejores score
      svc_cv.best_score_

[33]: 0.9800000000000001
```

Fuente: Angel Peña (2025).

se cargan nuevos datos de flores Iris que no formaron parte del entrenamiento, se extraen las características, y se utiliza el modelo SVM afinado con los mejores hiperparámetros para predecir sus clases. La predicción se imprime para visualizar los resultados de clasificación de forma inmediata, permitiendo comprobar cómo se comporta el modelo con datos reales fuera del entrenamiento.

Figura 14.

```
[36]: #Predecir nuevos valores
      df_new = pd.read_excel(r'C:\Users\king1\tareasPython\dataset practica guiada 5\iris_nuevos_datos.xlsx')
      X_new = df_new.values
      y_prediction = svc_cv.predict(X_new)
      print("Prediccion: {}".format(y_prediction))

      Prediccion: ['Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
       'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
       'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa'
       'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa']
```

Fuente: Angel Peña (2025).

Conclusión

Esta Práctica permitió consolidar los conocimientos sobre el ciclo completo del aprendizaje supervisado, aplicando modelos de clasificación sobre el dataset Iris y comprendiendo cómo evaluar, comparar y optimizar diferentes algoritmos para mejorar la precisión en la clasificación. El uso de GridSearchCV demostró ser una herramienta efectiva para encontrar los mejores hiperparámetros de modelos como KNN y SVM, elevando su desempeño de forma significativa. Al final de la práctica, se logró predecir correctamente la clase de nuevos registros de flores Iris, confirmando que el modelo puede generalizar correctamente sobre datos no vistos previamente. Esta práctica refuerza la importancia del preprocesamiento de datos, la selección de modelos y la optimización de parámetros en proyectos de minería de datos, habilidades fundamentales para la aplicación práctica de la ciencia de datos en contextos académicos y profesionales.